

Chapter 08

Programming Principles

Algorithms and Problem Solving

Mr. SOK Pongsametrey
metreysk@gmail.com

This week we connect algorithm design to Programming Principles like clarity, modularity, and maintainability, showing how step-by-step problem solving leads to efficient, reusable solutions.

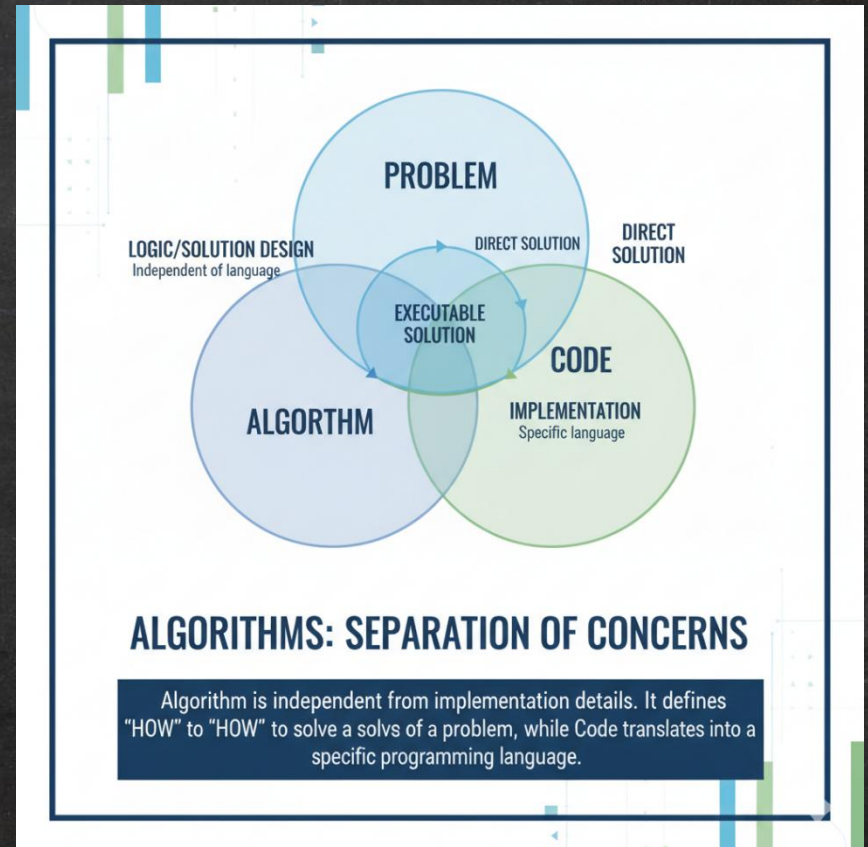
ABOUT

What is an Algorithm?

- **Definition:** A step-by-step procedure to solve a problem
- **Characteristics:**
 - **Input:** Zero or more inputs
 - **Output:** At least one output
 - **Definiteness:** Each step is precisely defined
 - **Finiteness:** Terminates after finite steps
 - **Effectiveness:** Basic operations are feasible
- **Real-world examples:** Cooking recipe, GPS navigation, search engines
- **Principle tie-in:** Clear, step-by-step algorithms improve code clarity and maintainability

Why Algorithm Thinking Matters

- Builds optimized, scalable software
- Essential in AI, data processing, search engines
- Core skill for coding interviews and problem-solving
- **Principle tie-in:** Separation of concerns — algorithm is independent from implementation details



Sorting Algorithms Overview

- Bubble Sort (simple, $O(n^2)$)
- Selection Sort
- Built-in: `Arrays.sort()`

```
int[] nums = {5, 3, 8, 1};  
Arrays.sort(nums);
```

- **Principle tie-in:** Use built-in, tested methods for reliability and maintainability (DRY)

Sort Algorithms

Use built-in, tested methods for reliability and maintainability (DRY - Don't Repeat Yourself)

UNSORTED LIST



SORTED LIST



SORTING ALGORITHMS: RELIABILITY AND MAINTAINABILITY

Use built-in, tested **“sort”** methods (DRY principle) to avoid bugs and improve code clarity.



Searching Algorithms Overview

- Linear Search: works on unsorted data
- Binary Search: fast, needs sorted array

```
int low = 0, high = arr.length - 1;
while (low <= high) {
    int mid = (low + high) / 2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) low = mid + 1;
    else high = mid - 1;
}
```

- **Principle tie-in:** Select the right algorithm for the data — KISS & efficiency

Search Algorithms

Select the right algorithm for efficiency (KISS)
(Keep It Simple & efficiency)



BINARY SEARCH TREES: EFFICIENT DATA RETRIEVAL

Selecting right algorithm (like BST for sorted data) improves search efficiency slow linear scans. KISS & be efficient!

Algorithm Design Process

1. **Understand the problem** - What exactly needs to be solved?
 2. **Identify inputs and outputs** - What data goes in/comes out?
 3. **Break down into smaller steps** - Divide and conquer approach
 4. **Choose appropriate data structures** - Arrays, lists, maps, etc.
 5. **Write pseudocode** - Language-independent solution
 6. **Implement in Java** - Convert to actual code
 7. **Test and optimize** - Verify correctness and improve efficiency
- **Principle tie-in: Systematic approach supports modularity and reduces complexity**

Time Complexity (Big O Notation)

Understanding algorithm efficiency:

Big O	Name	Example	Performance
$O(1)$	Constant	Array access <code>arr[5]</code>	Excellent
$O(\log n)$	Logarithmic	Binary search	Very Good
$O(n)$	Linear	Linear search	Good
$O(n \log n)$	Log-linear	Merge sort	Acceptable
$O(n^2)$	Quadratic	Bubble sort	Poor

- **Practical example:** Searching 1 million records: $O(1) = 1$ operation, $O(n) = 1$ million operations
- **Principle tie-in:** Understanding complexity helps choose efficient solutions

Linear Search Algorithm

```
public class LinearSearch {  
    /**  
     * Searches for a target value in an array  
     * Time Complexity: O(n)  
     * Space Complexity: O(1)  
     */  
    public static int linearSearch(int[] arr, int target) {  
        // Input validation  
        if (arr == null) {  
            throw new IllegalArgumentException("Array cannot be null");  
        }  
  
        // Search through each element  
        for (int i = 0; i < arr.length; i++) {  
            if (arr[i] == target) {  
                return i; // Found at index i  
            }  
        }  
        return -1; // Not found  
    }  
}
```

- **Principle tie-in: Simple, reliable algorithm that works on any dataset (KISS principle)**

Binary Search Algorithm

```
public class BinarySearch {  
    /**  
     * Searches for target in sorted array  
     * Time Complexity:  $O(\log n)$   
     * Space Complexity:  $O(1)$   
     * Prerequisite: Array must be sorted  
     */  
    public static int binarySearch(int[] arr, int target) {  
        if (arr == null) {  
            throw new IllegalArgumentException("Array cannot be null");  
        }  
  
        int left = 0;  
        int right = arr.length - 1;  
  
        while (left <= right) {  
            int mid = left + (right - left) / 2; // Avoid overflow  
  
            if (arr[mid] == target) {  
                return mid; // Found  
            } else if (arr[mid] < target) {  
                left = mid + 1; // Search right half  
            } else {  
                right = mid - 1; // Search left half  
            }  
        }  
        return -1; // Not found  
    }  
}
```

- **Principle tie-in: Efficient algorithm for sorted data - choosing right tool for the job**

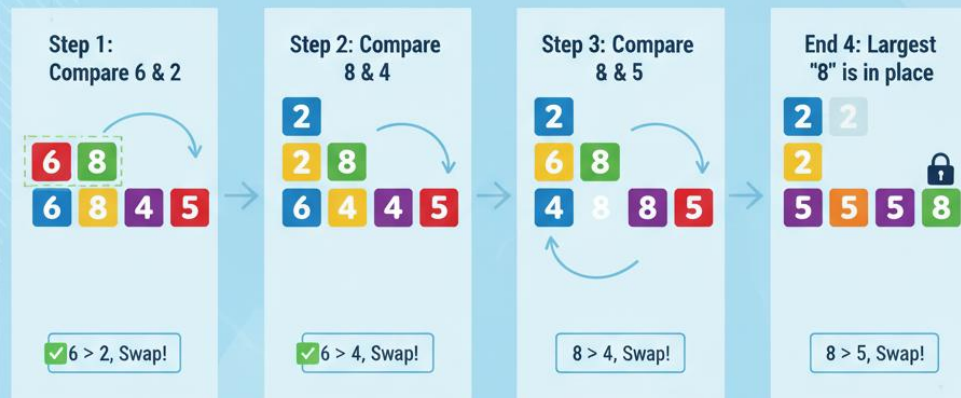
Bubble Sort Algorithm

```
public class BubbleSort {  
    /**  
     * Sorts array using bubble sort algorithm  
     * Time Complexity:  $O(n^2)$  - worst and average case  
     * Space Complexity:  $O(1)$   
     * Stable: Yes (maintains relative order of equal elements)  
     */  
    public static void bubbleSort(int[] arr) {  
        if (arr == null || arr.length < 2) {  
            return; // Already sorted or invalid  
        }  
  
        int n = arr.length;  
        boolean swapped;  
  
        for (int i = 0; i < n - 1; i++) {  
            swapped = false;  
  
            // Last i elements are already sorted  
            for (int j = 0; j < n - i - 1; j++) {  
                if (arr[j] > arr[j + 1]) {  
                    // Swap elements  
                    swap(arr, j, j + 1);  
                    swapped = true;  
                }  
            }  
  
            // If no swapping occurred, array is sorted  
            if (!swapped) {  
                break; // Optimization: early termination  
            }  
        }  
  
        private static void swap(int[] arr, int i, int j) {  
            int temp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = temp;  
        }  
    }  
}
```

- **Principle tie-in: Modular helper methods improve code reusability and readability**

Bubble Sort

Clear step-step logic supports learning and debugging



BUBBLE SORT: STEP-BY-STEP LOGIC

Visualizing each comparison and swap the algorithm clear, making easier to asir to learnn and debug. Clear logic prevents bugs!

Algorithm Comparison and Selection

When to use which algorithm:

Algorithm	Best for	Time Complexity	Space	Stable
Bubble Sort	Learning, small data	$O(n^2)$	$O(1)$	Yes
Selection Sort	Memory-constrained	$O(n^2)$	$O(1)$	No
Insertion Sort	Small/nearly sorted	$O(n^2)/O(n)$	$O(1)$	Yes
Binary Search	Sorted data search	$O(\log n)$	$O(1)$	-
Linear Search	Unsorted data	$O(n)$	$O(1)$	-

Built-in Java alternatives:

```
// Use Java's optimized sorting
Arrays.sort(arr); // TimSort -  $O(n \log n)$ 

// Use Java's search methods
int index = Arrays.binarySearch(sortedArr, target);
```

- **Principle tie-in: Choose the right tool for the job - balance simplicity vs. performance**

Real-world Algorithm Applications

Search Engines (Google, Bing):

- **PageRank algorithm** ranks web pages
- **Inverted index** for fast text search
- **Autocomplete** using trie data structures

Social Media (Facebook, Instagram):

- **News feed algorithm** shows relevant posts
- **Friend recommendation** using graph algorithms
- **Image recognition** using neural networks

E-commerce (Amazon, eBay):

- **Recommendation systems** suggest products
- **Inventory management** using optimization
- **Price comparison** using efficient search

AI and Machine Learning:

- **Natural Language Processing** algorithms
- **Computer vision** algorithms
- **Prediction algorithms** for decision making

Principle tie-in: Algorithms are the foundation of modern software systems

Problem-Solving with Algorithms

Example: Finding Duplicate Numbers

```
public class DuplicateFinder {  
    // Approach 1: Brute Force - O(n2)  
    public boolean hasDuplicatesBruteForce(int[] arr) {  
        for (int i = 0; i < arr.length; i++) {  
            for (int j = i + 1; j < arr.length; j++) {  
                if (arr[i] == arr[j]) {  
                    return true;  
                }  
            }  
        }  
        return false;  
    }  
  
    // Approach 2: Using HashSet - O(n)  
    public boolean hasDuplicatesOptimized(int[] arr) {  
        Set seen = new HashSet<>();  
        for (int num : arr) {  
            if (seen.contains(num)) {  
                return true;  
            }  
            seen.add(num);  
        }  
        return false;  
    }  
}
```

- **Principle tie-in: Multiple solutions exist - choose based on constraints and requirements**

Algorithm Performance Testing

```
public class AlgorithmBenchmark {
    public static void main(String[] args) {
        int[] sizes = {1000, 5000, 10000, 20000};

        for (int size : sizes) {
            int[] arr = generateRandomArray(size);

            System.out.println("Array size: " + size);

            // Test Bubble Sort
            int[] arr1 = arr.clone();
            long start = System.nanoTime();
            BubbleSort.bubbleSort(arr1);
            long bubbleTime = System.nanoTime() - start;

            // Test Built-in Sort
            int[] arr2 = arr.clone();
            start = System.nanoTime();
            Arrays.sort(arr2);
            long builtinTime = System.nanoTime() - start;

            System.out.printf("Bubble: %d ms, Built-in: %d ms%n",
                bubbleTime / 1_000_000, builtinTime / 1_000_000);
        }
    }
}
```

- **Principle tie-in: Empirical testing validates theoretical complexity analysis**

Lab Activity - Smart Search System

- **Build an intelligent search system that demonstrates multiple algorithms:**
- **Requirements:**
 - **Data Structure:** Create a Product class with name, category, price, rating
 - **Search Algorithms:** Implement multiple search strategies:
 - Linear search by name
 - Binary search by price (sorted)
 - Category filtering
 - Rating-based ranking
 - **Sorting Options:** Allow sorting by different criteria
 - **Performance Comparison:** Measure and compare algorithm performance
- **Focus: Apply DRY, modularity, and performance optimization principles**

```
Product[] products = {  
    new Product("Laptop", "Electronics", 999.99, 4.5),  
    new Product("Coffee Mug", "Kitchen", 15.99, 4.8),  
    new Product("Running Shoes", "Sports", 129.99, 4.2),  
    // ... more products  
};
```

Exercise: Leaderboard Management System

- Create a comprehensive leaderboard system for a gaming application
- Core Requirements:
 - Player Management:
 - Add new players with scores
 - Update existing player scores
 - Remove players from leaderboard
- Ranking Algorithms:
 - Sort players by score (descending)
 - Handle tie-breaking (by name alphabetically)
 - Maintain top N players efficiently
- Search Functionality:
 - Find player by name (linear search)
 - Find players within score range (binary search)
 - Search by rank position
- Performance Analysis:
 - Measure time complexity of operations
 - Compare different sorting algorithms
 - Analyze memory usage

Algorithm Optimization Strategies

1. Early Termination:

```
// Stop sorting if array becomes sorted
public void optimizedBubbleSort(int[] arr) {
    boolean swapped;
    for (int i = 0; i < arr.length - 1; i++) {
        swapped = false;
        for (int j = 0; j < arr.length - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr, j, j + 1);
                swapped = true;
            }
        }
        if (!swapped) break; // Already sorted
    }
}
```

2. Preprocessing for Better Performance:

```
// Sort once, search many times
public class OptimizedSearcher {
    private int[] sortedData;

    public OptimizedSearcher(int[] data) {
        this.sortedData = data.clone();
        Arrays.sort(this.sortedData); // O(n log n) once
    }

    public boolean search(int target) {
        return Arrays.binarySearch(sortedData, target) >= 0; // O(log n) per
search
    }
}
```

Principle tie-in: Optimization balances performance with code complexity

Algorithms in Modern Development

Industry Best Practices:

✅ Use Standard Library First:

- `Arrays.sort()` instead of custom sorting
- `Collections.binarySearch()` for searching
- `Stream.sorted()` for functional approach

📊 Profile Before Optimizing:

- Measure actual performance bottlenecks
- Use tools like JProfiler
- Don't optimize prematurely

🚀 Algorithm Selection Criteria:

- **Data size:** Small vs. large datasets
- **Data characteristics:** Sorted, random, duplicates
- **Performance requirements:** Speed vs. memory
- **Maintenance:** Simple vs. complex implementation

🌌 Future Trends:

- **Quantum algorithms** for specific problems
- **Machine learning** for algorithm selection
- **Parallel algorithms** for multi-core systems

Principle tie-in: Good algorithms are foundation for scalable, maintainable systems

Wrap-up and Q&A

- **Key Takeaways:**
 - **Algorithms are step-by-step solutions** to computational problems
 - **Time and space complexity** guide algorithm selection
 - **Different problems require different approaches**
 - **Built-in library methods** are usually optimized and tested
 - **Performance testing** validates theoretical analysis

Wrap-up and Q&A

- **Programming Principles Applied:**
 - **Clarity:** Well-structured algorithms are easier to understand
 - **Modularity:** Breaking algorithms into functions improves reusability
 - **Efficiency:** Choosing appropriate algorithms improves performance
 - **Maintainability:** Simple, documented algorithms are easier to modify
 - **DRY:** Reusing proven algorithms avoids reinventing the wheel

Wrap-up and Q&A

- **Real-world Relevance:**
 - **Technical interviews** heavily focus on algorithmic problem-solving
 - **System design** requires understanding algorithm trade-offs
 - **AI and ML** are built on sophisticated algorithms